Practice Question

Design a Turing Machine that accepts **all binary strings with an even number of 1's** (e.g., `00`, `1010`, `1111`) and rejects those with an odd number of 1's (e.g., `1`, `101`, `111`).

Tape alphabet: {start, 0, 1, Blank}

States: {q_init, q_even, q_odd, q_halt, q_rej}

Start_state: {q_init}

Accept: {q_halt}

Reject: {q_rej}

(q_init, start) → (q_start, start, R)

Even state:

(q_start, 0) → (q_even, 0, R)

(q_start, 1) → (q_odd, 1, R)

(q_start, Blank) → (q_halt, Blank, S)

(q_even, blank)→(q_halt, blank, S)

Odd state:

(q_odd, 0) → (q_odd, 0, R)

(q_odd, 1) → (q_even, 1, R)

(q_odd, Blank) → (q_halt, Blank, S)

We're building a TM for the language:

$$L = \{w \in \{0,1\}^* \mid w \text{ ends with } 0\}$$

So the machine must **accept** if the input ends in `0`, and **reject** otherwise.

Tape alphabet: {Start, 0, 1, Blank}

States: {q_init, q_0, q_last0, q_last1, q_halt, q_rej}

(q_init, start) → (q_start, Start, R)

(q_start, blank) → (q_rej, blank, S)

(q_start, 0) → (q_last0, 0, R)

(q_start, 1) → (q_last1, 1, R)

(q_last0, 0) → (q_last0, 0, R)

(q_last0, 1) → (q_last1, 1, R)

(q_last0, blank) → (q_halt, blank, S)

(q_last1, 0) → (q_last0, 0, R)

(q_last1, 1) → (q_last1, 1, R)

(q_last1, blank) → (q_rej, 1, S)

Practice Question 4

Design a deterministic Turing Machine (TM) that decides the language:

$$L = \{w \in \{0,1\}^* \mid w \text{ starts and ends with the same symbol}\}$$

Qfirst_0,0—qlast_0

Tape Alphabet: {start, 0, 1, blank}

States: {q_init, q_first0, q_first1, q_last0, q_last1, q_halt, q_rej}

(q_init, start) → (q_start, start, R)

(q_start, 0) → (q_first0, 0, R)

(q_start, 1) → (q_first1, 1, R)

(q_start, blank) → (q_rej, blank, S)

For q_start0

(q_first0, 0) → (q_last0, 0, R)

(q_first0, 1) → (q_last1, 1, R)

(q_last0, 0) → (q_last0, 0, R)

(q_last0, 1) → (q_last1, 1, R)

(q_last0, blank) → (q_halt, blank, S)

(q_last1, blank) → (q_rej, blank, S)

For q_start1

(q_first1, 1) → (q_last1, 1, R)

(q_first1, 0) → (q_last0, 0, R)

(q_last1, 1) → (q_last1, 1, R)

(q_last1, 0) → (q_last0, 0, R)

(q_last1, blank) → (q_halt, blank, S)

(q_last0, blank) → (q_rej, blank, S)

Great — let's build a simple, clear 1-tape deterministic Turing Machine that **decides**

$$\mathcal{L} = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 10\}.$$

Tape alphabet:{start, 0, 1, blank}

States:{q_start, q_init, q_rej, q_halt, q_seen1, qseen0, q_seen10}

(q_init, start)→(q_start, start, R)

(q_start, blank) →(q_rej, blank, S)

(q_start, 0)→(q_seen0, 0, R)

(q_start, 1) → (q_seen1, 1, R)

(q_seen0, 0)→(q_seen0, 0, R)

(q_seen0,1)→(q_seen1, 1,R)

(q_seen0, blank)→(q_rej, blank, S)

(q_seen1, 1)→(q_seen1,1,R)

(q_seen1, 0) →(q_seen10, 0, R)

(q_seen1, blank) →(q_rej, blank, S)

(q_seen10, 0)→(q_seen10, 0,R)

(q_seen10, 1)→(q_seen10, 1, R

(q_seen10, blank), (q_halt, blank, S)

101010

Construct a deterministic TM of the kind you prefer, which decides the following language:

$$\mathcal{L} = \{w \in \{0,1\}^* \mid w \text{ does not contain 01 as a substring}\}$$

Study the complexity of TM you have defined.

tape alphabet: {start, 0, 1, blank}

states: {q_init, q_scan, q_seen0, q_halt, q_rej}

(q_init, start) --> (q_scan, start, R)

(q_scan, 1) --> (q_scan, 1, R)

(q_scan, 0) --> (q_seen0, 0, R)

(q_scan, blank) --> (q_halt, blank, S)

(q_seen0, 1) --> (q_rej, 1, S)

(q_seen0, 0) --> (q_seen0, 0, R)

(q_seen0, Blank) --> (q_halt, blank, S)

Describe, formally or informally, a deterministic TM of the kind you prefer, which decides the following language:

L={w|w starts with 00 and ends with 1}

Examples of strings which are in L are 001 and 001011, while 01∉L. Study the complexity of TM you have defined.

Tape alphabet: {start, 0, 1, blank}

States: {q_init, q_start, q_first0, q_scan0, q_scan1, q_halt, q_rej}

(q_init, start) → (q_start, start, R)

(q_start, 0) → (q_first0, 0, R)

(q_start, 1) → (q_rej, 1, S)

(q_start, blank) → (q_rej, blank, S)

(q_first0, 1) → (q_rej, 1, S)

(q_first0, 0) → (q_scan0, 0, R)

(q_first0, Blank) → (q_rej, blank, S)

(q_scan0, 1) → (q_scan1, 1, R)

(q_scan0, 0) → (q_scan0, 0, R)

(q_scan0, blank) → (q_rej, 0, S)

(q_scan1, 0) → (q_scan0, 0, R)

(q_scan1, 1) → (q_scan1, 1, R)

(q_scan1, blank) → (q_halt, 1, S)

Construct a deterministic TM of the kind you prefer, which decides the following language:

$\mathcal{L} = \{ w \in \{0,1\}^* \ \ w \text{ starts with a 1 and ends with a 1} \}$

Study the complexity of TM you have defined.

Tape alphabet: {start, 0, 1, blank}

States: {q_init, q_start, q_first1, q_scan0, q_scan1, q_halt, q_rej}

(q_init, start) → (q_start, start, R)

(q_start, 0) → (q_rej, 0, S)

(q_start, 1) → (q_first1, 1, S)

(q_start, blank) → (q_rej, blank, S)

(q_first1, 0) → (q_scan0, 0, R)

(q_first1, 1) → (q_scan1, 1, R)

(q_first1, blank) → (q_halt, blank, S)

(q_scan0, 1) → (q_scan1, 1, R)

(q_scan0, 0) → (q_scan0, 0, R)

(q_scan0, blank) → (q_rej, blank, S)

(q_scan1, 0) → (q_scan0, 0, R)

(q_scan1, 1) → (q_scan1, 1, R)

(q_scan1, blank) → (q_halt, blank, S)

Design a Turing Machine that accepts all binary strings with an **odd number of 0's** (e.g., `1`, `011`, `10101`) and rejects those with an even number of 0's (e.g., `00`, `11`, `1001`).

Tape alphabet: {start, 0, 1 blank}

States: {q_init, start, q_halt, q_rej, q_even0, q_odd1}

(q_start, blank) → (q_rej, blank, S)

(q_start, 0) → (q_odd, 0, R)

(q_start, 1) →(q_even, 1, R)

Even:

(q_even,1) →(q_even, 1,R)

(q_even,0)→(q_odd,0,R)

(q_even, blank) → (q_rej, blank, S)

Odd

(q_odd, 1)→(q_odd,1,R)

(q_odd, 0) →(q_even,0, R)

(q_odd, blank) → (q_halt, blank, S)

📌 **Practice Question 2**

Design a Turing Machine that accepts all binary strings that **start and end with the same symbol** (e.g., `0`, `1`, `010`, `1011`) and rejects those that don't (e.g., `01`, `10`, `0110`).

Tape alphabet:{start, 0, 1, blank}

States: {q_init, q_start, q_halt, q_rej, q_st1, q_st0, q_en1, q_en0}

(q_init, start)→ (q_start, start, R)

(q_start, blank) →(q_rej, blank, S)

(q_start, 1) → (q_st1, 1, R)

(q_start, 0) → (q_st0, 0, R)

(q_st1, blank) → (q_halt, blank, S)

(q_st0, blank)→ (q_halt, blank, S)

(q_st1,1)→(q_en1, 1,R)

(q_st1,0)→(q_st1,0,R)

(q_en1, blank)→(q_halt, blank, S)

(q_st0, 0)→(q_en0, 0,R)

(q_st0, 1) → (q_en0, 1, R)

(q_en0, blank) → (q_halt, blank, S)